

Guide des concepts

Service, commande, agrégat, événement, réplica, projection : ce que c'est, à quoi ça sert, avec des exemples de notre projet de réseau social de concerts (OnY Va).

Ce document sert de référence à l'équipe pour la présentation, afin que nous comprenions tous la même chose. Les exemples sont des **exemples de travail** : ils évolueront avec le projet.

0. Vue d'ensemble : l'idée générale

Dans une application classique (un « monolithe »), il y a **une seule grosse base SQL** et tout le monde y accède. Ici, on fait l'inverse :

- l'application est découpée en **petits services** ;
- chaque service a **sa propre base de données**, du type le mieux adapté ;
- les services **ne lisent jamais directement les données des autres** : ils s'envoient des **événements** ;
- chaque service garde une **copie** des données dont il a besoin (un **réplica**).

Le but : l'autonomie. Si un service tombe en panne, les autres continuent de fonctionner.

```
Utilisateur --commande--> [ Service A ] --événement--> [ Service B ] --événement--> [ Service C ]
                        base A                        base B                        base C
                                                (réplica de A)                (projection)
```

1. Le service

C'est quoi ?

Un **petit programme indépendant** qui s'occupe d'un **seul domaine métier**, avec sa propre base de données.

À quoi ça sert ?

- Séparer les responsabilités : chacun sait précisément ce qu'il gère.
- Choisir la **bonne base pour chaque besoin** (document, graphe, etc.).
- Se répartir le travail à trois sans se marcher dessus.
- Modifier ou arrêter un service sans casser les autres.

Dans notre projet

Service	Ce qu'il gère	Base	Pourquoi cette base
Evenements	concerts, festivals, lieux	MongoDB	les fiches ont des champs variables
Social	comptes, amis, artistes suivis, « j'y vais »	Neo4j	ce sont des relations entre personnes
Feed	publications, photos, avis	MongoDB	chaque post est un bloc autonome
Calendrier	calendrier personnel	MongoDB	vue précalculée, prête à lire

Règle d'or : un service ne lit jamais directement la base d'un autre service.

2. La commande

C'est quoi ?

Une **demande de traitement** envoyée à un service : « fais telle action ». Elle exprime une **intention** et peut être refusée (par exemple si les données sont invalides). Les noms sont des **verbes à l'impératif** : CreerEvenement, AjouterAmi, PublierPost.

À quoi ça sert ?

C'est la **porte d'entrée** d'un service. Tout ce qui modifie des données passe par une commande, ce qui rend clair ce que chaque service sait faire.

Exemple

```
{
  "commande": "ReprogrammerEvenement",
  "service": "Evenements",
  "donnees": { "evenementId": "evt42", "nouvelleDate": "2027-09-05" }
}
```

Commande ou événement ?

	Commande	Événement
Sens	« Fais ça »	« Ça s'est passé »
Temps	impératif	passé
Peut être refusé ?	oui	non, c'est un fait
Destinataire	un seul service	plusieurs services possibles
Exemple	ReprogrammerEvenement	EvenementReprogramme

3. L'agrégat

C'est quoi ?

Un **bloc de données cohérent**, qu'on lit et qu'on écrit **en une seule fois**. C'est l'objet métier complet, avec tout ce qui lui appartient. Pensez à un **dossier** : au lieu d'éparpiller les informations dans 5 tables, on les met toutes dans le même dossier.

À quoi ça sert ?

- **Une seule lecture** suffit pour avoir tout l'objet (pas de JOIN).
- Les modifications d'un agrégat sont **cohérentes** : tout est écrit ou rien.
- C'est l'**unité naturelle** d'une base NoSQL orientée document.

Exemple : l'agrégat Evenement

```
{
  "_id": "evt42",
  "nom": "Rock en Seine",
  "type": "festival",
  "dates": ["2027-08-27", "2027-08-28"],
  "lieu": { "nom": "Domaine de Saint-Cloud", "ville": "Saint-Cloud" },
  "lineup": [
    { "artisteId": "a12", "nom": "Phoenix", "jour": "2027-08-28", "heure": "21:30" }
  ],
  "statut": "programme"
}
```

Le lieu et le line-up sont **dans** le document. En SQL, ce seraient 3 tables et 2 JOIN.

Comment le choisir ?

Posez-vous la question : « *Quelles données doivent toujours être lues ou modifiées ensemble ?* » Cela forme un agrégat. Un agrégat est **propre à un service** : l'agrégat Evenement appartient au service Evenements.

4. Pourquoi découpler les services ?

Imaginons que le service Feed doive afficher le **nom de l'événement** dans chaque post.

	Solution	Conséquence
Mauvaise (couplage)	À chaque affichage, Feed appelle le service Evenements pour demander le nom.	Si Evenements est en panne, Feed aussi. Chaque affichage charge Evenements.
Bonne (découplage)	Feed garde une copie du nom . Quand il change, Evenements envoie un événement et Feed met sa copie à jour.	Feed marche seul , même si Evenements est éteint : c'est l' autonomie .

5. Le réplica

C'est quoi ?

Une **copie partielle** des données d'un autre service, stockée dans la base du service qui en a besoin. « Partielle » : on ne copie **que ce qui est utile**, pas tout l'agrégat.

À quoi ça sert ?

- **Autonomie** : le service n'a pas besoin d'appeler les autres.
- **Disponibilité** : il continue de fonctionner si l'émetteur tombe.
- **Rapidité** : la donnée est locale, donc plus rapide à lire.

Exemple

L'agrégat complet dans **Evenements** (source de vérité) :

```
{ "_id": "evt42", "nom": "Rock en Seine", "type": "festival",
  "dates": ["2027-08-27", "2027-08-28"],
  "lieu": { "nom": "Domaine de Saint-Cloud", "ville": "Saint-Cloud" },
  "lineup": [ ... ], "createurId": "u10", "statut": "programme" }
```

Le réplica dans **Social** (juste le nécessaire pour « j'y vais » et « amis participants ») :

```
{ "id": "evt42", "nom": "Rock en Seine", "date": "2027-08-27", "ville": "Saint-Cloud", "statut": "programme" }
```

Le réplica dans **Feed** (juste de quoi afficher un post) :

```
{ "id": "evt42", "nom": "Rock en Seine" }
```

Vocabulaire à connaître

- **Source de vérité** : le service qui possède la donnée d'origine (ici Evenements). Lui seul peut la modifier.
- Un réplica est **en lecture seule** pour le service qui le reçoit : on ne le modifie que via les événements.

Le prix à payer : la cohérence éventuelle

*Entre l'instant où l'événement change et celui où les réplicas sont à jour, il peut s'écouler **quelques instants** pendant lesquels les copies sont différentes. On l'accepte : c'est la **cohérence éventuelle**. Compromis typique du NoSQL : on gagne en autonomie et en vitesse, on perd en cohérence immédiate.*

6. L'événement

C'est quoi ?

Un **message qui annonce qu'un fait s'est produit** dans un service. Il est écrit au **passé** : `EvenementCree`, `EvenementReprogramme`, `UtilisateurRenomme`. Il ne peut pas être refusé : c'est déjà arrivé.

À quoi ça sert ?

À **prévenir les autres services** pour qu'ils mettent à jour leurs réplicas ou leurs projections, sans que l'émetteur ait besoin de les connaître.

Ce que doit contenir un événement (selon le sujet)

Élément	Question	Exemple
Service émetteur	Qui l'envoie ?	Evenements
Service récepteur	Qui le reçoit ?	Social
Donnée de l'agrégat modifiée	Quoi a changé ?	date
Nouvelle valeur	Quelle est la valeur maintenant ?	2027-09-05

```
{
  "evenement": "EvenementReprogramme",
  "emetteur": "Evenements",
  "recepteur": "Social",
  "donnee": "date",
  "nouvelleValeur": "2027-09-05"
}
```

Les deux types d'événements à définir

- **Création du réplica** : envoie toutes les données utiles pour que le récepteur crée sa copie (ex. `EvenementCree`).
- **Mise à jour du réplica** : envoie uniquement ce qui a changé (ex. `EvenementReprogramme`).

Comment les événements circulent-ils ?

Dans une vraie architecture, on utilise un **broker de messages** (RabbitMQ, Kafka, ou même Redis Pub/Sub) : le service émetteur **publie** l'événement, les services intéressés s'y **abonnent**. Pour un projet de cours, on peut aussi les simuler plus simplement (voir la section 13).

7. La projection

C'est quoi ?

Une **vue de données construite à partir d'événements et de réplicas, transformée** et préparée pour un **usage précis**. Elle est stockée dans un **nouveau service**. C'est comme préparer un **résumé** : au lieu de recalculer à chaque fois, on garde le résultat prêt à lire.

À quoi ça sert ?

- Répondre à une **question précise** avec **une seule lecture**, sans calcul ni JOIN.
- Croiser des données de **plusieurs services** sans les interroger en direct.
- Ne pas surcharger les services d'origine.

Exemple : le service Calendrier

Besoin : « afficher le calendrier d'un utilisateur avec, pour chaque concert, les amis qui y vont ». Les données sont réparties : les événements sont dans **Evenements**, les participations et les amis dans **Social**. Le service Calendrier **écoute** `EvenementCree`, `EvenementReprogramme` (`Evenements`) et `ParticipationAjoutée` (`Social`), et construit un agrégat prêt à lire :

```
{
  "_id": "u1",
  "evenements": [
    { "evenementId": "evt42", "nom": "Rock en Seine", "date": "2027-08-27",
      "ville": "Saint-Cloud", "amisParticipants": ["Tom", "Sarah"] }
  ]
}
```

La requête devient triviale : `db.calendriers.findOne({ _id: "u1" })`

Réplica ou projection ?

	Réplica	Projection
Contenu	copie (partielle) de l'agrégat d'un service	transformation de données de plusieurs sources
Objectif	autonomie du service	répondre à un nouveau besoin
Structure	proche de l'original	différente , pensée pour la lecture
Exemple	l'événement dans Social	le calendrier d'un utilisateur

8. Les types de bases NoSQL utilisés

Famille	Exemple	Principe	Dans notre projet
Document	MongoDB	stocke des objets JSON complets	Evenements, Feed, Calendrier
Graphe	Neo4j	stocke des nœuds et des relations	Social (amis, suivis, participations)
Clé-valeur	Redis	associe une clé à une valeur, très rapide	non utilisé (bonus possible : cache)
Colonnes	Cassandra	gros volumes de données écrites en continu	non utilisé

Pourquoi le graphe pour Social ?

La question « mes amis **et leurs amis** qui vont à ce concert » est une **suite de relations**. En SQL, elle demande des jointures qui se multiplient. En graphe, c'est une requête courte :

```
MATCH (moi:Utilisateur {id: "u1"})-[:AMI_DE*1..2]-(ami)-[:PARTICIPE]->(e:Evenement {id: "evt42"})
RETURN DISTINCT ami.pseudo
```

C'est **votre meilleur argument** pour justifier le NoSQL devant le prof.

9. SQL vs NoSQL en résumé

	SQL	NoSQL
Structure	tables liées	documents, graphes, clés-valeurs...
Schéma	fixe	souple
Données liées	JOIN	imbriquées ou dupliquées
Doublons	à éviter (normalisation)	volontaires (dénormalisation)
Cohérence	forte (ACID)	souvent éventuelle
Montée en charge	surtout verticale	surtout horizontale
Idéal pour	données très structurées, transactions	données variées, gros volumes, lecture rapide

Le principe clé : en SQL, on organise les données selon leur **structure**. En NoSQL, on les organise selon **la façon dont l'application les lit**.

10. Le parcours d'une donnée : exemple complet

Scénario : un artiste reprogramme un concert.

1. L'artiste envoie la commande ReprogrammerEvenement au service **Evenements**.
2. Evenements modifie la date dans son agrégat (MongoDB) : c'est la **source de vérité**.
3. Evenements publie l'événement EvenementReprogramme avec la nouvelle date.
4. **Social** le reçoit et met à jour son **réplica** de l'événement.
5. **Feed** le reçoit et met à jour son réplica (si la date y est utilisée).

6. **Calendrier** le reçoit et met à jour sa **projection** : le concert apparaît à la nouvelle date dans le calendrier des utilisateurs concernés.
7. À aucun moment un service n'a lu la base d'un autre.

Pendant tout ce temps, si le service Evenements s'arrête, Social, Feed et Calendrier continuent de répondre avec leurs copies.

11. Glossaire express

Terme	Définition en une phrase
Service	petit programme autonome avec sa propre base, dédié à un domaine
Commande	demande d'action adressée à un service (impératif)
Agrégat	bloc de données cohérent, lu et écrit en une fois
Événement	message annonçant qu'un fait s'est produit (passé)
Réplica	copie partielle de données d'un autre service, pour l'autonomie
Projection	vue transformée, construite pour un nouveau besoin
Source de vérité	le service qui possède la donnée d'origine
Autonomie	capacité d'un service à fonctionner sans les autres
Découplage	les services ne dépendent pas directement les uns des autres
Dénormalisation	duplication volontaire de données pour lire plus vite
Cohérence éventuelle	les copies finissent par être identiques, avec un léger délai
Broker	outil qui transporte les événements (RabbitMQ, Kafka...)

12. Questions que le prof peut poser

Pourquoi dupliquer les données ? C'est mal en SQL.

En NoSQL, c'est volontaire : on gagne en autonomie et en vitesse de lecture, en acceptant une cohérence éventuelle.

Que se passe-t-il si un événement est perdu ?

Le réplica devient obsolète. En production, un broker garantit la livraison et permet de rejouer les événements. Pour le projet, on peut le citer comme limite.

Pourquoi plusieurs bases différentes ?

Chaque type de donnée a un besoin différent : relations pour Social (graphe), fiches variables pour Evenements (document). Choisir la bonne base simplifie les requêtes.

Quelle est la différence entre réplica et projection ?

Le réplica copie un agrégat pour l'autonomie. La projection transforme et croise des données pour répondre à un nouveau besoin.

Comment prouver l'autonomie ?

En démo : arrêter le service Evenements et montrer que Social, Feed et Calendrier continuent de fonctionner.

13. Pour la mise en pratique

Pas besoin d'une infrastructure lourde pour un projet de cours :

- **Bases** : MongoDB et Neo4j via **Docker** (une ligne de commande chacune).
- **Services** : des petits scripts (Python avec Flask ou FastAPI, ou Node.js) qui exposent les commandes.
- **Événements**, par ordre de difficulté croissante : (1) des fichiers JSON échangés pour simuler, (2) Redis Pub/Sub, simple à installer, (3) RabbitMQ, plus proche de la réalité.
- **Démo** : un scénario qui montre la création d'un événement, la propagation du réplica, puis la panne d'un service.

Commencez par un **scénario minimal** (créer un événement, le répliquer dans Social, l'afficher dans le Calendrier) avant d'ajouter le reste.